EPICODE

Esame SQL Rev 1

Saraceni Barbara

Questions EPICODE

01. Cosa si intende per database?

Un **database** è una raccolta organizzata di dati strutturati, gestiti digitalmente. In un database relazionale, le informazioni sono memorizzate in tabelle (che rappresentano le entità), composte da colonne (campi o attributi) e righe (record). La caratteristica principale è che queste tabelle possono essere messe in relazione tra loro attraverso chiavi specifiche, permettendo di gestire grandi volumi di dati in modo efficiente, garantendo l'integrità ed evitando la ridondanza delle informazioni.

02. Cos'è un DBMS?

Un **DBMS (Database Management System)** è un software che permette di gestire un database e funge da interfaccia tra l'utente e i dati archiviati. Il suo compito principale è consentire la creazione, la manipolazione e l'amministrazione dei dati in modo sicuro ed efficiente. Senza un DBMS, non potremmo interrogare i dati tramite il linguaggio SQL. Un esempio pratico è MySQL, il sistema di gestione di database relazionali che stiamo utilizzando nel corso.

03. Indica le principali clausole di uno statement SELECT in ordine di esecuzione logica. Descrivi per ciascuna delle clausole indicate la logica di funzionamento.

L'ordine logico di esecuzione è diverso da quello di scrittura. SQL segue questi passaggi:

- **FROM:** È il primo passo. Il sistema individua la tabella (o le tabelle tramite JOIN) da cui deve prelevare i dati. Se non sappiamo dove sono i dati, non possiamo fare altro.
- **WHERE:** Serve a filtrare le singole righe della tabella. SQL scarta subito i record che non soddisfano i criteri scelti, così lavora su un set di dati più piccolo.
- **GROUP BY:** Questa clausola serve a mettere insieme i record che hanno valori uguali in determinate colonne. È utile quando non vogliamo vedere i singoli dettagli, ma dei riepiloghi (come il totale o la media).
- **HAVING:** Funziona come un "WHERE" ma per i gruppi. Serve a filtrare i risultati dopo che sono stati raggruppati dalla GROUP BY.
- **SELECT:** Solo a questo punto SQL sceglie quali colonne mostrare effettivamente nel risultato finale. Qui vengono calcolate anche le funzioni di aggregazione (come SUM o AVG).
- **ORDER BY:** È l'ultimo passaggio. Prende il risultato finale e lo ordina (crescente o decrescente) per facilitare la lettura all'utente.

04. Descrivi, immaginando uno scenario a te familiare, il concetto di group by. Utilizza l'approccio che ritieni più efficiente per trasmettere il concetto (suggerimento: disegna anche una sola tabella in Excel o in word con poche colonne e pochi record e descrivi, basandosi sulla tabella stessa, un esempio di group by).

Il **GROUP BY** è la clausola che ci permette di passare dal dettaglio del singolo record a un dato aggregato, unendo i record che condividono lo stesso valore in una colonna specifica.
Per spiegarlo tramite un esempio, scelgo uno studio sociologico in Calabria sulla spesa dei trasporti per gli studenti universitari. Immaginiamo di avere questa tabella su Excel, chiamata Indagine_Calabria:

Questions EPICODE

ID_Studente	Nome	Quartiere	Spesa_Trasporti
1	Pasquale	Lido	40
2	Maria	Santa Caterina	15
3	Giuseppe	Arcavacata	10
4	Francesca	Lido	45
5	Antonio	Arcavacata	12

Se il mio obiettivo è capire quanto si spende in media per ogni zona, la sintassi SQL sarebbe:

```
SELECT Quartiere, AVG(Spesa_Trasporti) AS Spesa_Media
FROM Indagine_Calabria
GROUP BY Quartiere;
```

SQL "raggruppa" i record e restituisce questa tabella di sintesi:

Quartiere	Spesa_Media
Lido	42.5
Santa Caterina	15
Arcavacata	11

Come si vede dalle tabelle, il sistema ha preso i record di "Lido" (40 e 45) e ne ha calcolato la media, facendo lo stesso per gli altri quartieri. In questo modo, trasformiamo una lista di singoli individui in un report sintetico che permette di confrontare l'andamento del fenomeno tra i diversi gruppi.

05. Descrivi la differenza tra uno schema OLTP e uno schema OLAP

- **OLTP (Online Transactional Processing)** è usato per gestire i dati giornalieri, come inserire nuovi record o aggiornare informazioni. I dati sono divisi in tante tabelle separate (normalizzate), per evitare errori e duplicati. Ad esempio, potresti avere una tabella per gli studenti e un'altra per i quartieri. Per vedere tutto insieme, il sistema deve spesso "unire" le tabelle usando le query.
- **OLAP (Online Analytical Processing)** è usato per analizzare grandi quantità di dati, fare statistiche o calcoli. I dati sono organizzati in modo "piatto" (denormalizzato), un po' come se tutte le tabelle fossero già unite in una sola grande tabella. Questo rende le interrogazioni molto più veloci, perché il database non deve collegare tante tabelle diverse.

06. Dato un medesimo scenario di analisi, qual è la differenza in termini di risultato ottenibile tra una join e una subquery?

La differenza principale è nel contenuto e nel numero di righe del risultato:
La **JOIN** restituisce un risultato che unisce le colonne di entrambe le tabelle. Se ci sono più corrispondenze, il record principale viene ripetuto (moltiplicazione delle righe). Si usa quando vogliamo vedere i dettagli di entrambi i database.
La **SUBQUERY** restituisce solo le colonne della tabella principale. Viene usata come filtro e non ripete mai i record, anche se ci sono molte corrispondenze nella seconda tabella. Si usa quando vogliamo sapere se un dato esiste, senza però visualizzare i dettagli dell'altra tabella.

Questions EPICODE

07. Cosa si intende per DML e DDL?

DDL (Data Definition Language): Sono i comandi che servono per definire la struttura del database. Si usano quando dobbiamo creare la tabella o cambiare la sua forma.

- Comandi principali: CREATE (per creare la tabella), ALTER (per modificarne la struttura), DROP (per cancellare tutta la tabella).

DML (Data Manipulation Language): Sono i comandi che servono per lavorare con i dati e i contenuti che stanno dentro le tabelle. Si usano per gestire le informazioni che abbiamo raccolto.

- Comandi principali: SELECT (per leggere i dati), INSERT (per inserire nuovi dati), UPDATE (per modificare i dati esistenti), DELETE (per cancellare i dati).

08. Quali istruzioni possono utilizzare per estrarre l'anno da un campo data? Proponi degli esempi.

Prima di scegliere la funzione, è fondamentale identificare il tipo di dato utilizzato nel database, poiché ognuno presenta limiti diversi:

- DATE: Memorizza solo la data nel formato YYYY-MM-DD. Il range è compreso tra 1000-01-01 e 9999-12-31.
- DATETIME: Memorizza data e ora nel formato YYYY-MM-DD hh:mm:ss. Il range è compreso tra 1000-01-01 00:00:00 e 9999-12-31 23:59:59.
- TIMESTAMP: Memorizza data e ora nel formato YYYY-MM-DD hh:mm:ss, ma con un range più limitato: dal 1970-01-01 00:00:01 UTC al 2038-01-19 03:14:07 UTC

Per estrarre l'anno da un campo data in SQL, si possono utilizzare le seguenti istruzioni:

Funzione YEAR: Questa funzione restituisce un valore numerico intero che rappresenta l'anno di una data specifica.

Funzione EXTRACT: Questa istruzione permette di isolare l'unità temporale desiderata (YEAR) da un campo che contiene dati più complessi, come DATETIME o TIMESTAMP.

09. Qual è la differenza tra gli operatori logici AND e OR?

La differenza principale risiede nel numero di condizioni che devono essere soddisfatte affinché un record venga incluso nei risultati della query:

Operatore AND (Vero + Vero = Vero)

L'operatore AND è restrittivo: restituisce un record solo se tutte le condizioni specificate sono contemporaneamente vere.

- Esempio: SELECT * FROM Product WHERE Categoria = 'Bikes' AND Prezzo > 100; (Questo comando mostrerà solo i prodotti che appartengono alla categoria Bikes e che, allo stesso tempo, hanno un prezzo superiore a 100).

Operatore OR (Vero + Falso = Vero)

L'operatore OR è inclusivo: restituisce un record se almeno una delle condizioni specificate è vera.

- Esempio: SELECT * FROM Product WHERE Categoria = 'Bikes' OR Categoria = 'Clothing'; (Questo comando mostrerà tutti i prodotti che appartengono alla categoria Bikes, più tutti quelli che appartengono alla categoria Clothing).

Questions EPICODE

10. È possibile innestare una query nella clausola SELECT?

Sì, è possibile inserire una **subquery** nella clausola SELECT. Questa deve restituire un singolo valore scalare (un solo dato) per ogni riga della query principale.

Esempio: Se volessi visualizzare il nome di ogni prodotto e, in una colonna accanto, il numero totale di vendite registrate nel database:

```
SELECT ProductName, (SELECT COUNT(*) FROM Sales) AS NumeroTotaleVendite
FROM Product;
```

11. Qual è la differenza tra l'operatore logico OR e l'operatore logico IN?

L'operatore logico **OR** restituisce un record se almeno una delle condizioni specificate è vera. Può combinare condizioni diverse che riguardano anche colonne differenti. Invece, l'operatore logico **IN** viene utilizzato per verificare se un valore corrisponde a uno qualsiasi degli elementi presenti in un elenco predefinito.

- Esempio OR:

```
SELECT *
FROM Product
WHERE Categoria = 'Bikes' OR Categoria = 'Clothing';
```

- Esempio IN:

```
SELECT *
FROM Product
WHERE Categoria IN ('Bikes', 'Clothing', 'Accessories');
```

12. L'operatore logico BETWEEN include anche gli estremi del range specificato?

Sì, l'operatore BETWEEN include sempre gli estremi del range.

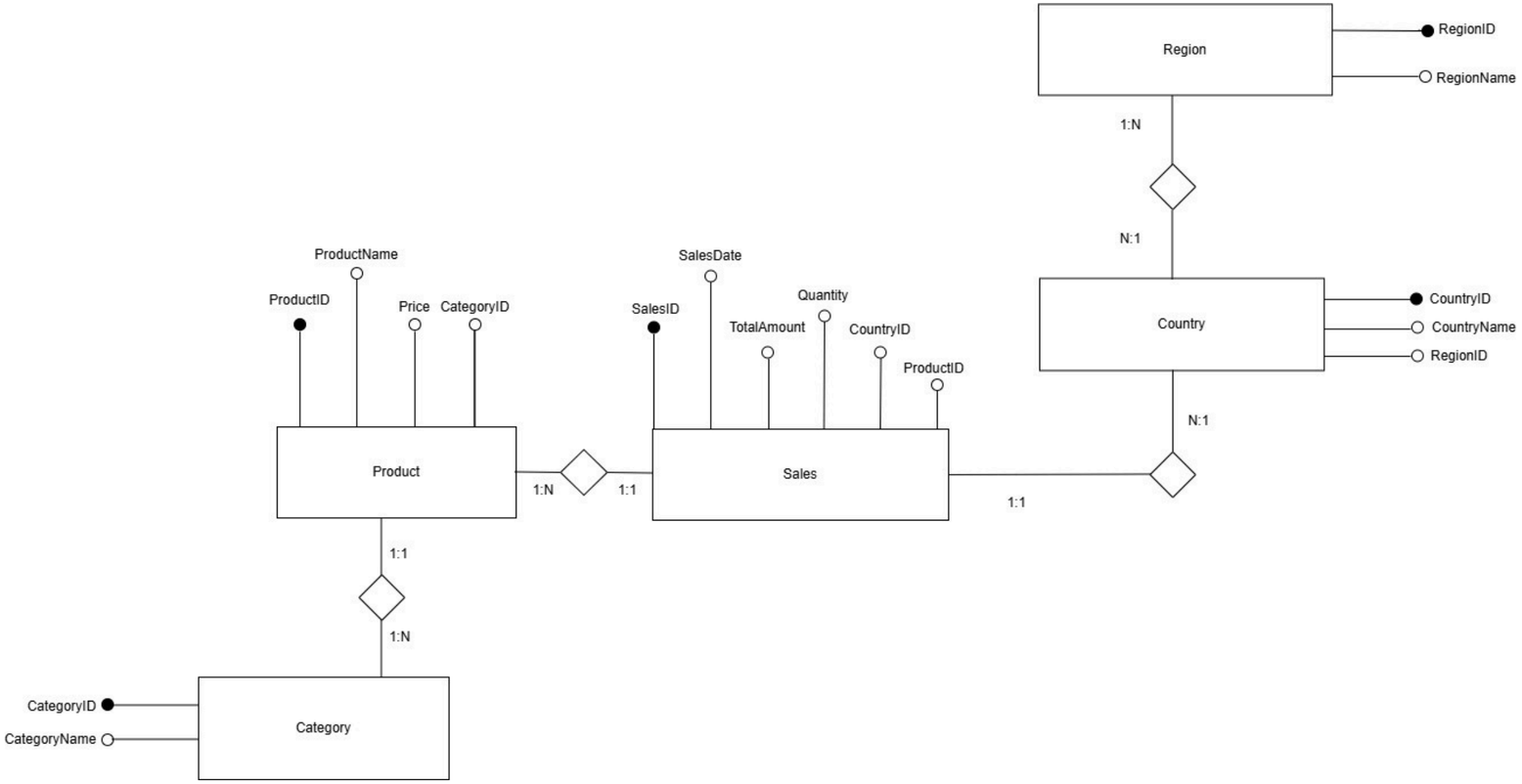
- Esempio:

```
SELECT *
FROM Studenti
WHERE Età BETWEEN 18 AND 25;
```

Questa query seleziona tutti gli studenti la cui età è compresa tra 18 e 25, inclusi 18 e 25.

Case Study EPICODE

Task 1: Proponi una progettazione concettuale e logica della base dati
La progettazione concettuale deve includere tutte le entità coinvolte e le relazioni tra queste. Per ciascuna entità indica l'attributo chiave e i principali attributi descrittivi (non è necessario indicare tutti gli attributi).



Realizzato su: https://app.diagrams.net/

Case Study EPICODE

Task 2: Descrivi la struttura delle tabelle che reputi utili e sufficienti a modellare lo scenario proposto tramite la sintassi DDL. Implementa fisicamente le tabelle utilizzando il DBMS SQL Server (o altro).

Product

ProductID (PK)	ProductName	Price	CategoryID (FK)
1	Set Mattoncini Castello	45	1
2	Gioco dell'Oca	15,50	2

1 : 1

Category

CategoryID (PK)	CategoryName
1	Costruzioni
2	Giochi da Tavolo

1 : N

Sales

SalesID (PK)	SalesDate	TotalAmount	Quantity	CountryID (FK)	ProductID (FK)
1	2/20/2026	90	2	1	1
2	2/21/2026	30	2	2	2

N : 1

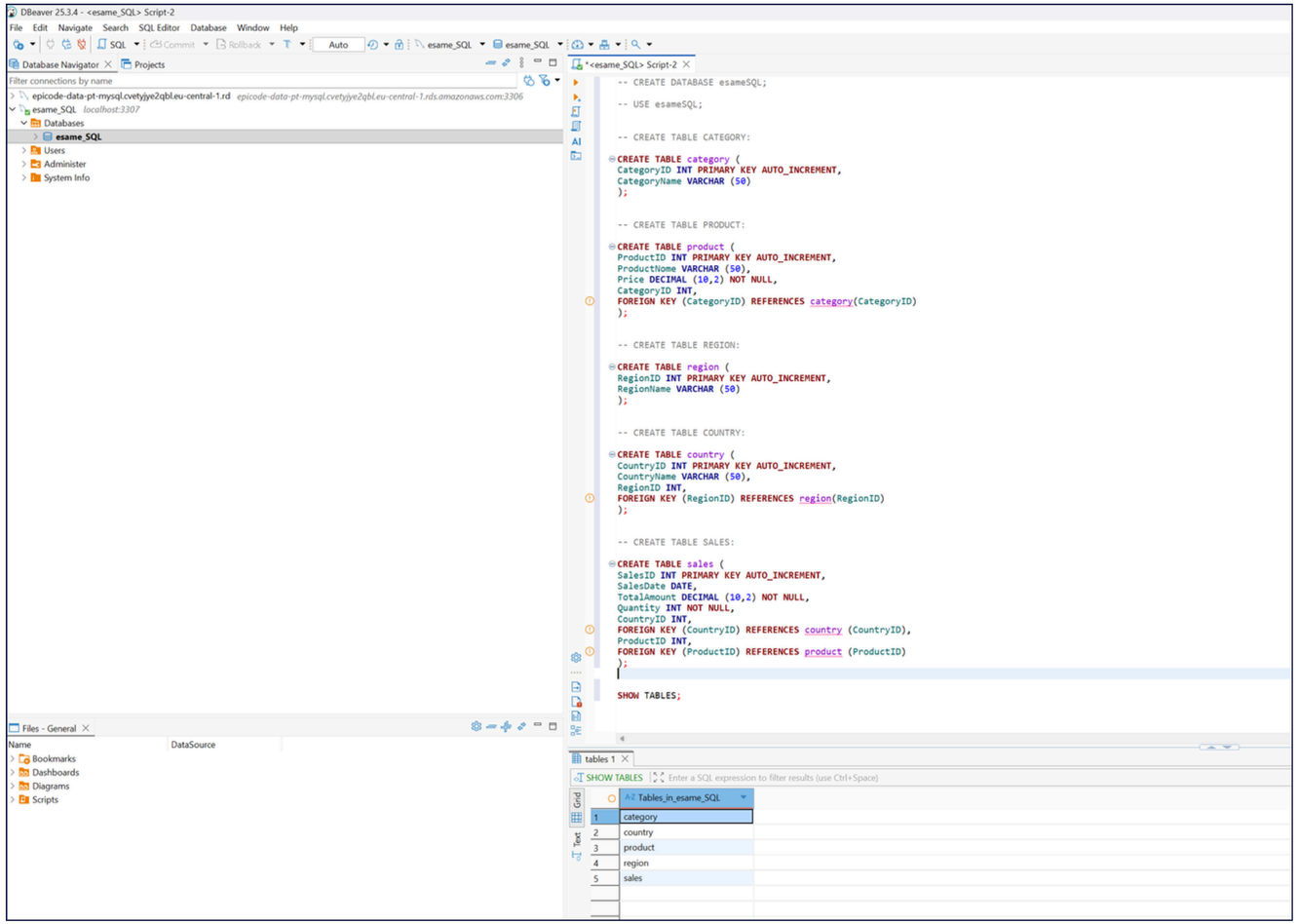
Country

CountryID (PK)	CountryName	RegionID (FK)
1	Italia	1
2	Stati Uniti	2

N : 1

Region

RegionID (PK)	RegionName
1	Europa
2	Nord America



Case Study EPICODE

Task 3: Popola le tabelle utilizzando dati a tua discrezione (sono sufficienti pochi record per tabella; riporta le query utilizzate)

Popolazione Category:

```sql
INSERT INTO category (CategoryName)
VALUES ('Elettronica'),
       ('Arredamento'),
       ('Giocattoli');
--
SELECT * FROM category;
--
UPDATE category
SET CategoryName = 'Abbigliamento'
WHERE CategoryID = 2;
--
UPDATE category
SET CategoryName = 'Sport'
WHERE CategoryID = 3;
--
SELECT * FROM category;
```

category 1

SELECT * FROM category | Enter a SQL expression to filter results (use Ctrl+Space)

	CategoryID	CategoryName
1	1	Elettronica
2	2	Abbigliamento
3	3	Sport
4	4	Arredamento
5	5	Giocattoli

Popolazione Region:

```sql
INSERT INTO region (RegionName)
VALUES
    ('Europa'),
    ('NordAmerica'),
    ('Asia'),
    ('SudAmerica'),
    ('Oceania')
    ;

SELECT * FROM region;
```

region 1

SELECT * FROM region | Enter a SQL expression to filter results (use Ctrl+Space)

	RegionID	RegionName
1	1	Europa
2	2	NordAmerica
3	3	Asia
4	4	SudAmerica
5	5	Oceania

Popolazione Country:

```sql
INSERT INTO country (CountryID, CountryName, RegionID)
VALUES
    (1, 'Italia', 1),
    (2, 'Canada', 2),
    (3, 'Giappone', 3),
    (4, 'Brasile', 4),
    (5, 'Australia', 5);

SELECT * FROM country;
```

country 1

SELECT * FROM country | Enter a SQL expression to filter results (use Ctrl+Space)

	CountryID	CountryName	RegionID
1	1	Italia	1
2	2	Canada	2
3	3	Giappone	3
4	4	Brasile	4
5	5	Australia	5

Case Study EPICODE

Task 3: Popola le tabelle utilizzando dati a tua discrezione (sono sufficienti pochi record per tabella; riporta le query utilizzate)

Popolazione Product:

```sql
INSERT INTO product (ProductID, ProductNome, Price, CategoryID)
VALUES
    (101, 'Smartphone Z', 899.99, 1),
    (201, 'Sedia Ufficio', 150.00, 2),
    (301, 'Lego Castle', 89.90, 3),
    (401, 'Giacca Invernale', 120.50, 4),
    (501, 'Racchetta Tennis', 75.00, 5);

SELECT * FROM product;
```

product 1

SELECT * FROM product | Enter a SQL expression to filter results (use Ctrl+Space)

	ProductID	ProductNome	Price	CategoryID
1	101	Smartphone Z	899.99	1
2	201	Sedia Ufficio	150	2
3	301	Lego Castle	89.9	3
4	401	Giacca Invernale	120.5	4
5	501	Racchetta Tennis	75	5

Popolazione Sales:

```sql
INSERT INTO sales (SalesID, SalesDate, TotalAmount, Quantity, CountryID, ProductID)
VALUES
    (1, '2026-01-10', 899.99, 1, 1, 101),
    (2, '2026-01-12', 300.00, 2, 2, 201),
    (3, '2026-01-15', 89.90, 1, 3, 301),
    (4, '2026-01-20', 120.50, 1, 4, 401),
    (5, '2026-01-25', 150.00, 2, 5, 501),
    (6, '2026-01-11', 899.99, 1, 1, 101),
    (7, '2026-01-26', 150.00, 2, 5, 501),
    (8, '2026-01-28', 150.00, 2, 5, 501),
    (9, '2026-01-20', 120.50, 1, 4, 401),
    (10, '2026-01-20', 120.50, 1, 4, 401),
    (11, '2025-01-20', 120.50, 1, 4, 401),
    (12, '2025-01-19', 120.50, 1, 4, 401);

SELECT * FROM sales;
```

sales 1

SELECT * FROM sales | Enter a SQL expression to filter results (use Ctrl+Space)

	SalesID	SalesDate	TotalAmount	Quantity	CountryID	ProductID
1	1	2026-01-10	899.99	1	1	101
2	2	2026-01-12	300	2	2	201
3	3	2026-01-15	89.9	1	3	301
4	4	2026-01-20	120.5	1	4	401
5	5	2026-01-25	150	2	5	501
6	6	2026-01-11	899.99	1	1	101
7	7	2026-01-26	150	2	5	501
8	8	2026-01-28	150	2	5	501
9	9	2026-01-20	120.5	1	4	401
10	10	2026-01-20	120.5	1	4	401
11	11	2025-01-20	120.5	1	4	401
12	12	2025-01-19	120.5	1	4	401

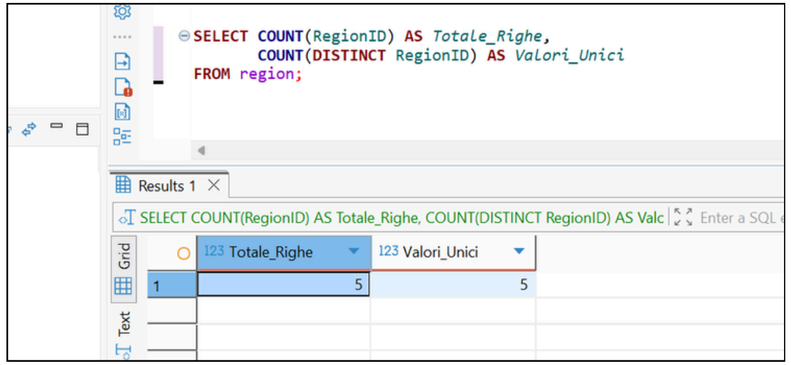
Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

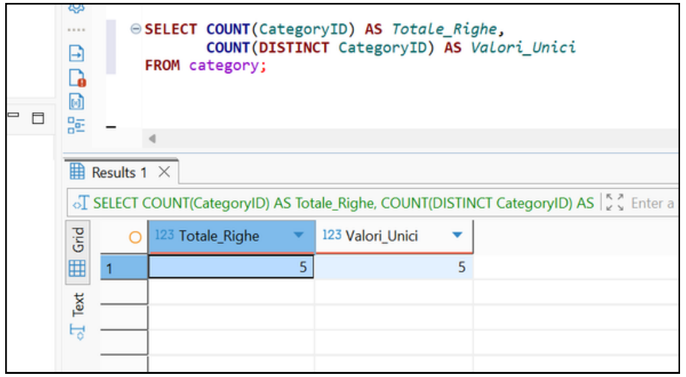
1)Verificare che i campi definiti come PK siano univoci. In altre parole, scrivi una query per determinare l'univocità dei valori di ciascuna PK (una query per tabella implementata).

Ho scelto di utilizzare il confronto tra COUNT() e COUNT(DISTINCT) come metodo di verifica dell'univocità: se **Totale_Righe è uguale a Valori_Unici**, la PK è confermata come univoca.

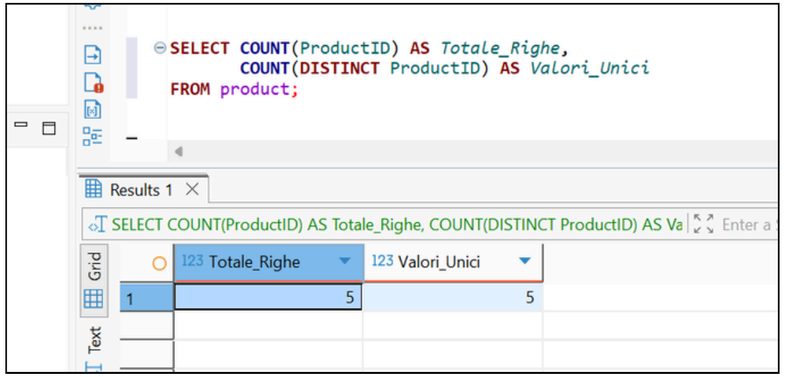
Region:



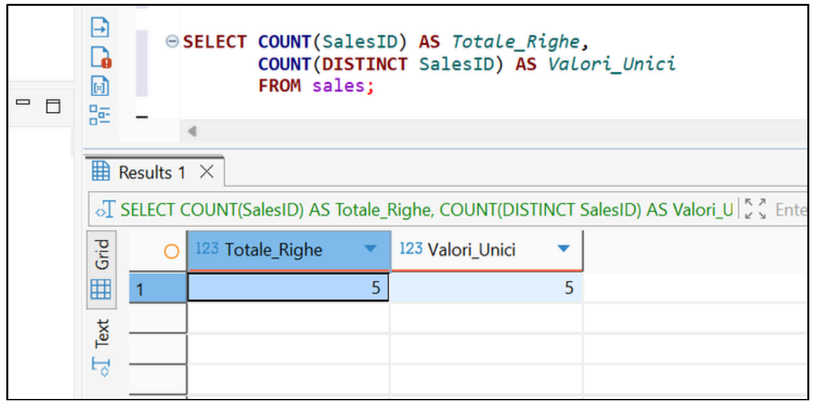
Category:



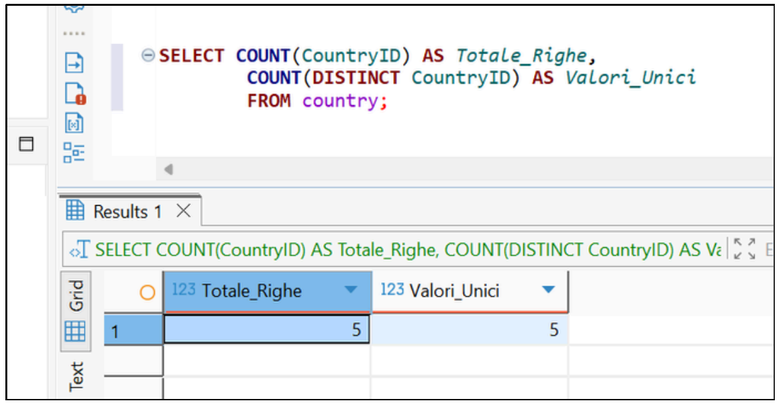
Product



Sales:



Country:



Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

2)Esporre l'elenco delle transazioni indicando nel result set il codice documento, la data, il nome del prodotto, la categoria del prodotto, il nome dello stato, il nome della regione di vendita e un campo booleano valorizzato in base alla condizione che siano passati più di 180 giorni dalla data vendita o meno (>180 -> True, <= 180 -> False)

```sql
SELECT
    s.SalesID AS codice_documento,
    s.SalesDate AS data,
    p.ProductNome AS nome_del_prodotto,
    c.CategoryName AS categoria_del_prodotto,
    co.CountryName AS stato,
    r.RegionName AS nome_della_regione,
    CASE
        WHEN DATEDIFF(CURDATE(), s.SalesDate) > 180 THEN 'True'
        ELSE 'False'
    END AS PiuDi180Giorni
FROM sales AS s
JOIN product p ON s.ProductID = p.ProductID
JOIN category c ON p.CategoryID = c.CategoryID
JOIN country co ON s.CountryID = co.CountryID
JOIN region r ON co.RegionID = r.RegionID;
```

sales(+) 1

SELECT s.SalesID AS codice_documento, s.SalesDate AS data, p.ProductNome A | Enter a SQL expression to filter results (use Ctrl+Space)

	codice_documento	data	nome_del_prodotto	categoria_del_prodotto	stato	nome_della_regione	PiuDi180Giorni
1	1	2026-01-10	Smartphone Z	Elettronica	Italia	Europa	False
2	2	2026-01-12	Sedia Ufficio	Abbigliamento	Canada	NordAmerica	False
3	3	2026-01-15	Lego Castle	Sport	Giappone	Asia	False
4	4	2026-01-20	Giacca Invernale	Arredamento	Brasile	SudAmerica	False
5	5	2026-01-25	Racchetta Tennis	Giocattoli	Australia	Oceania	False

Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

3) Esporre l'elenco dei prodotti che hanno venduto, in totale, una quantità maggiore della media delle vendite realizzate nell'ultimo anno censito. (ogni valore della condizione deve risultare da una query e non deve essere inserito a mano). Nel result set devono comparire solo il codice prodotto e il totale venduto.

```
-- 1) Media delle vendite realizzate nell'ultimo anno censito:
SELECT s.ProductID, s.Quantity
FROM sales AS s
WHERE s.Quantity > (SELECT AVG(Quantity) AS media_vendite
FROM sales
-- 2) Nell'ultimo anno censito:
-- 3) Esporre l'elenco dei prodotti:
WHERE YEAR(SalesDate) = YEAR(CURDATE()));
```

sales 1

SELECT s.ProductID, s.Quantity FROM sales AS s WHERE s.Quantity > (SELECT A

Enter a SQL expression to filter results (use Ctrl+Space)

	ProductID	Quantity
1	201	2
2	501	2
3	501	2
4	501	2

Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

4) Esporre l'elenco dei soli prodotti venduti e per ognuno di questi il fatturato totale per anno.

```sql
-- 1) fatturato totale per anno
SELECT p.ProductNome, SUM(s.TotalAmount) AS totale_venduto , YEAR(s.SalesDate) AS anno
FROM product AS p
INNER JOIN sales AS s ON p.ProductID = s.ProductID
GROUP BY ProductNome, anno;
```

product 1

SELECT p.ProductNome, SUM(s.TotalAmount) AS totale_venduto , YEAR(s.Sales[Enter a SQL expression to filter results (use Ctrl+Space)

	ProductNome	totale_venduto	anno
1	Smartphone Z	1799.98	2026
2	Sedia Ufficio	300	2026
3	Lego Castle	89.9	2026
4	Giacca Invernale	361.5	2026
5	Giacca Invernale	241	2025
6	Racchetta Tennis	450	2026

Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

5) Esporre il fatturato totale per stato per anno. Ordina il risultato per data e per fatturato decrescente.

```sql
-- Esporre il fatturato totale per stato per anno. Ordina il risultato per data e per fatturato decrescente.

SELECT c.CountryName, SUM(TotalAmount) AS totale, YEAR(SalesDate) AS anno
FROM country AS c
INNER JOIN sales AS s ON c.CountryID = s.CountryID
GROUP BY CountryName, anno
ORDER BY anno DESC, totale DESC;
```

country 1

SELECT c.CountryName, SUM(TotalAmount) AS totale, YEAR(SalesDate) AS anno

Enter a SQL expression to filter results (use Ctrl+Space)

	CountryName	totale	anno
1	Italia	1799.98	2026
2	Australia	450	2026
3	Brasile	361.5	2026
4	Canada	300	2026
5	Giappone	89.9	2026
6	Brasile	241	2025

Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

6) Rispondere alla seguente domanda: qual è la categoria di articoli maggiormente richiesta dal mercato?

La categoria di articoli maggiormente richiesta dal mercato è Giocattoli.

```
-- qual è la categoria di articoli maggiormente richiesta dal mercato?
SELECT c.CategoryName AS category, SUM(s.Quantity) AS totale_quantita
FROM product AS p
INNER JOIN sales AS s ON p.ProductID = s.ProductID
INNER JOIN category AS c ON p.CategoryID = c.CategoryID
GROUP BY category
ORDER BY totale_quantita DESC;
```

category 1

SELECT c.CategoryName AS category, SUM(s.Quantity) AS totale_quantita FROM

	category	totale_quantita
1	Giocattoli	6
2	Arredamento	5
3	Elettronica	2
4	Abbigliamento	2
5	Sport	1

```
-- qual è la categoria di articoli maggiormente richiesta dal mercato?
SELECT c.CategoryName AS category, SUM(s.Quantity) AS totale_quantita
FROM product AS p
INNER JOIN sales AS s ON p.ProductID = s.ProductID
INNER JOIN category AS c ON p.CategoryID = c.CategoryID
GROUP BY category
ORDER BY totale_quantita DESC
LIMIT 1;
```

category 1

SELECT c.CategoryName AS category, SUM(s.Quantity) AS totale_quantita FROM

	category	totale_quantita
1	Giocattoli	6

Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

7) Rispondere alla seguente domanda: quali sono i prodotti invenduti? Proponi due approcci risolutivi differenti.

I prodotti invenduti sono: Smartphone Y, e Smartphone D.

Primo approccio:

```sql
-- Rispondere alla seguente domanda: quali sono i prodotti invenduti? Proponi due approcci risolutivi differenti

SELECT p.ProductNome, s.Quantity
FROM product AS p
LEFT JOIN sales AS s ON s.ProductID = p.ProductID
WHERE Quantity IS NULL;
```

product(+) 1

SELECT p.ProductNome, s.Quantity FROM product AS p LEFT JOIN sales AS s O

Enter a SQL expression to filter results (use Ctrl+Space)

	ProductNome	Quantity
1	Smartphone Y	[NULL]
2	Smartphone D	[NULL]

Secondo approccio:

```sql
SELECT p.ProductNome
FROM product AS p
WHERE ProductID NOT IN (SELECT DISTINCT ProductID FROM sales);
```

product 1

SELECT p.ProductNome FROM product AS p WHERE ProductID NOT IN (SELECT

Enter a SQL expression to filter results (use Ct

	ProductNome
1	Smartphone Y
2	Smartphone D

Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

8) Creare una vista sui prodotti in modo tale da esporre una “versione denormalizzata” delle informazioni utili (codice prodotto, nome prodotto, nome categoria)

```
-- Creare una vista sui prodotti in modo tale da esporre una “versione denormalizzata” delle informazioni utili (codice prodotto, nome prodotto, nome categoria)

CREATE VIEW versione_denormalizzata AS
SELECT p.ProductID, p.ProductNome, c.CategoryName
FROM product AS p
INNER JOIN category AS c ON c.CategoryID = p.CategoryID;

SELECT * FROM versione_denormalizzata;
```

versione_denormalizzata 1

SELECT * FROM versione_denormalizzata | Enter a SQL expression to filter results (use Ctrl+Space)

	ProductID	ProductNome	CategoryName
1	101	Smartphone Z	Elettronica
2	601	Smartphone Y	Elettronica
3	701	Smartphone D	Elettronica
4	201	Sedia Ufficio	Abbigliamento
5	301	Lego Castle	Sport
6	401	Giacca Invernale	Arredamento
7	501	Racchetta Tennis	Giocattoli

Value

101

Case Study EPICODE

Task 4: Dopo aver popolate le tabelle, scrivi delle query utili a:

9) Creare una vista per le informazioni geografiche

```sql
CREATE VIEW informazioni_geografiche AS
SELECT c.CountryName, r.RegionName
FROM country AS c
INNER JOIN region AS r ON c.RegionID = r.RegionID;

SELECT * FROM informazioni_geografiche;
```

informazioni_geografiche 1

SELECT * FROM informazioni_geografiche | Enter a SQL expression to filter results (use Ctrl+Space)

	CountryName	RegionName
1	Italia	Europa
2	Canada	NordAmerica
3	Giappone	Asia
4	Brasile	SudAmerica
5	Australia	Oceania

EPICODE

Grazie mille!

Saraceni Barbara